Zero Knowledge Proof

Dr. Ahlad Kumar
Blockchain Fundamentals

NFSU Gandhinagar

October 4, 2025

Dr. Ahlad Kumar

Outline

1. Informal Definition
2. Core Properties
3. Why the Properties Matter
4. Problem Statement
5. Overview
6. Setup
7. Ranges and Recommended Sizes
8. Why verification equations hold

Dr. Ahlad Kumar

What is a Zero-Knowledge Proof? (Informal)

- A **Zero-Knowledge Proof (ZKP)** is a protocol in which a **Prover** convinces a **Verifier** that a statement is true *without revealing any additional information* beyond the truth of the statement.
- Intuition: "I can open the locked door (prove I know the password) without telling you the password."
- Use cases: private transactions, anonymous credentials, zk-rollups, private voting.

Dr. Ahlad Kumar

Core Properties of a ZKP

A zero-knowledge proof must satisfy three key properties:

Completeness: If the statement is true and both parties follow the protocol, the verifier accepts (with high probability).

Soundness: If the statement is false, no cheating prover can convince the honest verifier except with negligible probability.

Zero-Knowledge: The verifier learns nothing beyond the truth of the statement; formally, there exists a simulator that can produce the verifier's view without access to the secret.

Dr. Ahlad Kumar

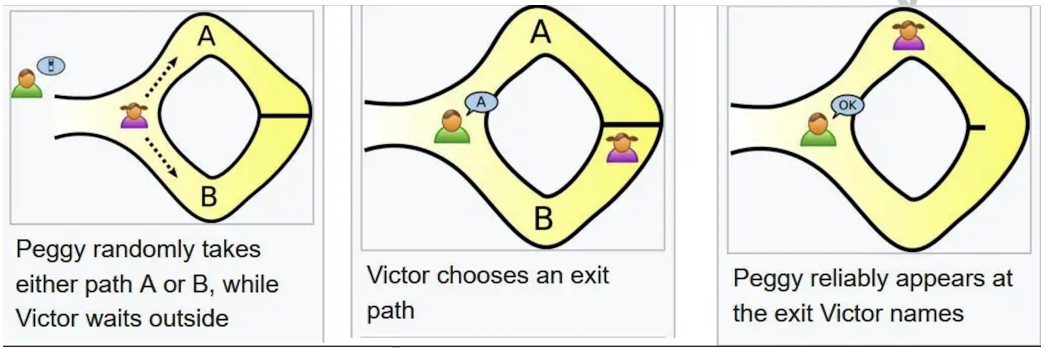
Why These Properties Matter

- **Completeness** ensures honest provers succeed (usability).
- **Soundness** prevents forgery (security).
- **Zero-Knowledge** protects privacy (confidentiality of secrets).

A well-designed ZKP balances these three: make cheating infeasible while revealing nothing sensitive.

Dr. Ahlad Kumar

Classical Cave Illustration



Peggy (Prover) chooses a random path, Victor (Verifier) challenges, and Peggy reliably appears at the requested exit.

Setup & Problem

- A circular cave with two paths (Left and Right) leading to a locked door in the center.
- The door can only be opened with the **secret password** known to the Prover (P).
- The Verifier (V) wants proof that P knows the secret **without learning the password** itself.
- Key Challenge: How to convince without revealing knowledge? This is the essence of Zero-Knowledge Proof.

Dr. Ahlad Kumar

Step-by-Step Process

1. Prover (P) enters the cave through either the Left or Right path at random.

Dr. Ahlad Kumar

Step-by-Step Process

1. Prover (P) enters the cave through either the Left or Right path at random.
2. Verifier (V) waits outside, not knowing which path P took.

Dr. Ahlad Kumar

Step-by-Step Process

1. Prover (P) enters the cave through either the Left or Right path at random.
2. Verifier (V) waits outside, not knowing which path P took.
3. V then calls out: *"Come out from the Left"* or *"Come out from the Right"*.

Dr. Ahlad Kumar

Step-by-Step Process

1. Prover (P) enters the cave through either the Left or Right path at random.
2. Verifier (V) waits outside, not knowing which path P took.
3. V then calls out: *"Come out from the Left"* or *"Come out from the Right"*.
4. If P knows the password, they can open the door in the middle and exit through whichever side V demands.

Dr. Ahlad Kumar

Step-by-Step Process

1. Prover (P) enters the cave through either the Left or Right path at random.
2. Verifier (V) waits outside, not knowing which path P took.
3. V then calls out: *"Come out from the Left"* or *"Come out from the Right"*.
4. If P knows the password, they can open the door in the middle and exit through whichever side V demands.
5. If P does not know the password, they can only return the same way they entered — they succeed only if V "guessed" their entry path.

Dr. Ahlad Kumar

Step-by-Step Process

1. Prover (P) enters the cave through either the Left or Right path at random.
2. Verifier (V) waits outside, not knowing which path P took.
3. V then calls out: *"Come out from the Left"* or *"Come out from the Right"*.
4. If P knows the password, they can open the door in the middle and exit through whichever side V demands.
5. If P does not know the password, they can only return the same way they entered — they succeed only if V "guessed" their entry path.
6. Repeating this experiment many times makes it nearly impossible for a fake prover to keep succeeding by chance.

Dr. Ahlad Kumar

What is the Fiat–Shamir Identification Protocol?

- A *challenge–response* identification protocol allowing a prover to convince a verifier that they **know a secret** s without revealing s.
- Security is *probabilistic*; multiple rounds amplify confidence.
- Widely used as an educational example of zero-knowledge identification schemes.

Dr. Ahlad Kumar

Public Parameters and Keys (Setup)

- Choose two (large) primes p, q. Compute modulus $n = p \cdot q$.
- Prover selects secret $s \in \mathbb{Z}_n^*$ (i.e., $1 < s < n$ and $\gcd(s, n) = 1$).
- Prover computes public verification key:

$$v \leftarrow s^2 \bmod n$$

- Publish: (n, v). Keep s secret.

Dr. Ahlad Kumar

Variables and Recommended Ranges / Sizes

Symbol	Meaning / Range / Recommendation
p, q	Large primes; used only to generate $n = p \cdot q$.
n	Modulus $= p \cdot q$. Typical size: 2048 bits (or 3072 bits) for classical security.
s	Secret; $s \in \mathbb{Z}_n^*$ (i.e., $1 < s < n$ and $\gcd(s, n) = 1$).
v	Public key: $v \equiv s^2 \pmod{n}$. (Published.)
r	Random nonce per-round; $r \in Z_n^*$ (uniform).
x	Commitment $x \equiv r^2 \pmod{n}$ (sent by prover).
e	Challenge bit $e \in \{0, 1\}$ (verifier chooses uniformly).
y	Response: $y = r$ (if $e = 0$) or $y = r \cdot s \bmod n$ (if $e = 1$).
t	Number of rounds; choose so $(1/2)^t$ is negligible (e.g., $t \geq 20$).

Notes: In practice use secure randomness and large modulus sizes.

Dr. Arvind Kumar

Fiat–Shamir Identification Protocol (Setup + One Round)

Fiat–Shamir Identification (setup + one round)

1: **Setup:**
2: Choose large primes p, q; compute $n \leftarrow p \cdot q$.
3: Prover chooses secret $s \in \mathbb{Z}_n^*$ (coprime to n).
4: Compute public key $v \leftarrow s^2 \bmod n$.
5: Publish (n, v); keep s secret.
6: **Identification Round:**
7: Prover picks random $r \in \mathbb{Z}_n^*$; computes $x \leftarrow r^2 \bmod n$; sends x.
8: Verifier chooses random challenge bit $e \in \{0, 1\}$; sends e.
9: Prover replies: $y \leftarrow r \cdot s^e \bmod n$.
10: Verifier checks: accept iff $y^2 \equiv x \cdot v^e \pmod{n}$.

Dr. Ahlad Kumar

Why the Verifier's Check Works

Assume the usual protocol relations (all arithmetic mod n):

$$x = r^2, \qquad v = s^2, \qquad y = r \cdot s^e \quad (e \in \{0, 1\}).$$

Then

$$y^2 = (r \cdot s^e)^2 = r^2 \cdot s^{2e} = x \cdot (s^2)^e = x \cdot v^e \pmod{n}.$$

Because the equality above holds for both possible values of e (0 or 1), the verifier's unified check

$$y^2 \equiv x \cdot v^e \pmod{n}$$

is satisfied whenever the prover computes $y = r \cdot s^e$ correctly.

Dr. Ahlad Kumar

Protocol Setup (Public)

- Choose two primes: $p = 3$, $q = 5 \quad \Rightarrow \quad n = p \cdot q = 15$ (public)
- Prover's secret: $s = 4$ (must be coprime with n)
- Public value: $v = s^2 \bmod n = 4^2 \bmod 15 = 16 \bmod 15 = 1$

Public parameters: $n = 15$, $v = 1$ **Secret:** $s = 4$

Dr. Ahlad Kumar

Round 1 — Honest Prover (Alice knows s, Case $e = 1$)

Steps (with numbers):

1. Alice chooses random $r = 2$.
2. Compute and send: $x = r^2 \bmod n = 2^2 \bmod 15 = 4$.
3. Verifier (Bob) sends challenge $e = 1$.
4. Alice computes response: $y = r \cdot s \bmod n = 2 \cdot 4 \bmod 15 = 8$.
5. Bob verifies:

$$y^2 \bmod n = 8^2 \bmod 15 = 64 \bmod 15 = 4$$

$$x \cdot v^e \bmod n = 4 \cdot 1^1 \bmod 15 = 4$$

Since both equal, verification **passes**.

Dr. Ahlad Kumar

Round 2 — Honest Prover (Alice knows s, Case $e = 0$)

Steps (with numbers):

1. Alice chooses random $r = 2$.
2. Compute and send: $x = r^2 \bmod n = 2^2 \bmod 15 = 4$.
3. Verifier (Bob) sends challenge $e = 0$.
4. Alice computes response: $y = r \bmod n = 2$.
5. Bob verifies:

$$y^2 \bmod n = 2^2 \bmod 15 = 4$$
$$x \cdot v^e \bmod n = 4 \cdot 1^0 \bmod 15 = 4$$

Since both equal, verification **passes**.

Dr. Ahlad Kumar

Round 3 — Dishonest Prover (Eve doesn't know s)

1. Eve picks a random $r = 2$, sends $x = r^2 \bmod n = 4$.
2. Bob sends challenge $e = 1$.
3. Eve does not know s, so she must guess a response. Suppose she guesses $y = 5$.
4. Verifier checks:

$$y^2 \bmod n = 5^2 \bmod 15 = 25 \bmod 15 = 10$$

$$x \cdot v^e \bmod n = 4 \cdot 1^1 \bmod 15 = 4$$

5. Since $10 \neq 4$, verification **fails**. Eve was caught.

Teaching point: a dishonest prover can succeed at most with probability $1/2$ per round; repeating reduces cheating probability to $(1/2)^t$ after t rounds.

Dr Ahlad Kumar

Summary / Class Exercise

- **Exercise for students:** repeat the example with different small primes (e.g., $p = 5, q = 7$) and pick a new s that is coprime with n. Show one honest and one dishonest round on the board.
- **Discussion:** Why do we repeat rounds? How does repeated verification reduce the probability of successful cheating? Compute the cheating probability after 10 rounds.
- **Note:** In real systems, p, q are very large (hundreds of bits) — this small-number example is only for pedagogy.

Dr. Ahlad Kumar

Byzantine Generals Problem and Blockchain Consensus

Dr. Ahlad Kumar
Blockchain Fundamentals

NFSU Gandhinagar

October 4, 2025

Dr. Ahlad Kumar

Byzantine Generals Problem — Definition

1. A classic problem in distributed computing and game theory.
2. Describes the difficulty of reaching consensus when some nodes are unreliable or malicious.
3. Key idea: how to ensure all loyal participants agree on one decision.

Dr. Ahlad Kumar

Byzantine Generals Problem — Notes

- **Problem Context:** In distributed systems, some participants (nodes) may be **unreliable, faulty, or malicious**. Goal: Reach agreement (**consensus**) despite these issues.
- **Story Analogy:** Several Byzantine generals surround a city.
  - They must all agree to either **Attack** or **Retreat**.
  - Communication only via messengers (no direct trust).
  - Some generals may be **traitors** who send false or conflicting orders.
- **Core Challenge:** Ensure that all **loyal generals** make the same decision, even if traitors exist.
- **Solution Requirements:**
  1. **Agreement:** All loyal generals decide the same plan.
  2. **Validity:** If the commander is loyal, all follow the commander's order.
  3. **Fault Tolerance:** Consensus possible if $n \geq 3f + 1$ (where f = max traitors).
- **Importance:** Foundation of consensus in distributed systems and blockchains (e.g., PBFT, Proof-of-Work, Proof-of-Stake).

Dr. Ahlad Kumar

Key Requirements for a Solution

1. **Agreement**: All loyal generals must decide the same plan.
2. **Validity**: If the commander is loyal, his order must be followed.
3. **Fault Tolerance**: Must work even if up to 1/3 of nodes are traitors.

Dr. Ahlad Kumar

Case 1: All Loyal (Consensus Trivial)

Setup: $n = 4$, $f = 0$ (no traitors). Generals: Alice (commander, honest), Bob (honest), Carol (honest), Dave (honest).

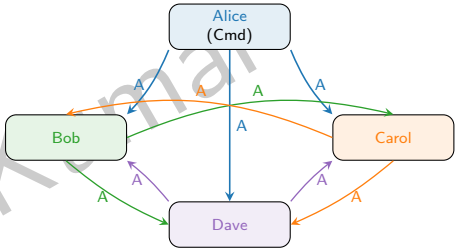
Run (compact):

1. Alice (commander) sends **A** to Bob, Carol, Dave.
2. Bob, Carol, Dave all forward the same A honestly.
3. Every lieutenant collects identical messages.

Vectors (order = [Alice,Bob,Carol,Dave]):

- **Bob:** [A, A, A, A] → all A.
- **Carol:** [A, A, A, A] → all A.
- **Dave:** [A, A, A, A] → all A.

Conclusion: All honest generals decide **Attack** without conflict — consensus trivially achieved.



All generals loyal. All messages = A (Attack).
Consensus trivial: everyone agrees.

Dr. Ahlad Kumar

Case 2: One Traitor (Consensus Achieved)

Setup: $n = 4$, tolerate $f = 1$. Generals: Alice (commander, honest), Bob (honest), Carol (honest), Dave (traitor).

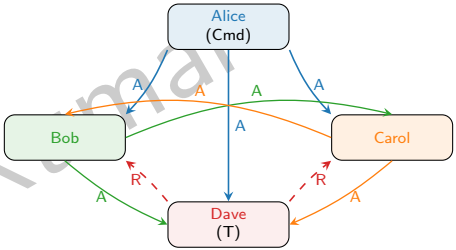
Run (compact):

1. Alice → everyone: **A**.
2. Bob, Carol forward A honestly.
3. Dave (traitor) sends R to confuse.
4. Honest lieutenants take majority of received messages.

Vectors (order = [Alice,Bob,Carol,Dave]):

- **Bob:** [A, A, A, R] ⇒ A=3 ⇒ **Attack**.
- **Carol:** [A, A, A, R] ⇒ A=3 ⇒ **Attack**.
- **Dave:** sees same vector but may act arbitrarily.

Conclusion: Honest lieutenants (Bob & Carol) both decide **Attack** — consensus achieved.



Colors: Alice, Bob, Carol, Dave.
Solid arrows = honest forwards (A); dashed = traitor false forwards (R

Dr. Ahlad Kumar

Case 3: Two Traitors (Consensus Fails)

Setup: $n = 4$, two traitors. Generals: Alice (commander, honest), Bob (honest), Carol (traitor), Dave (traitor).

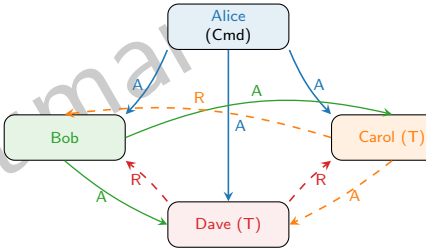
Run (compact):

1. Alice → everyone: **A**.
2. Bob forwards A honestly.
3. Carol (traitor) and Dave (traitor) forward conflicting messages to cause disagreement.
4. Each lieutenant takes majority of received messages (if any).

Vectors (order = [Alice,Bob,Carol,Dave]):

- **Bob:** [A, A, R, R] ⇒ A=2, R=2 ⇒ **Tie (no majority)**.
- **Carol (traitor):** [A, A, R, R] (may decide arbitrarily).
- **Dave (traitor):** [A, A, R, R] (may decide arbitrarily).
- **Alice (commander):** knows she sent A to all.

Conclusion: Honest lieutenant (Bob) sees a tie and cannot deterministically decide the same as others → **No guaranteed consensus with two traitors**.



Colors: Alice, Bob, Carol (T), Dave (T).
Solid arrows = honest forwards (A); dashed = traitor forwards (possib

Dr. Ahlad Kumar

Why Important?

1. Core challenge in distributed systems and blockchain.
2. In a network, some nodes may crash, fail, or act maliciously.
3. Consensus ensures all honest nodes agree on the same ledger state.
4. Blockchain protocols (PoW, PoS, PBFT) are practical solutions.

Dr. Ahlad Kumar

In Short

1. Byzantine Generals Problem = achieving trust and agreement.
2. Even if some participants lie, fail, or behave maliciously.

Dr. Ahlad Kumar

Mapping to Blockchain

1. Generals $\rightarrow$ Nodes (miners/validators).
2. Messages $\rightarrow$ Transactions/blocks.
3. Traitors $\rightarrow$ Malicious nodes (double-spend, equivocation).
4. Goal: all honest nodes agree on the same on the same blockchain history.

Dr. Ahlad Kumar

Bitcoin Proof of Work — Mechanism

1. Miners compete to solve a cryptographic puzzle (hash $<$ target).
2. Winner proposes a block and broadcasts it.
3. Other nodes verify the proof and transactions.
4. If valid, they accept and extend that chain.
5. In case of forks, follow the longest (most work) chain.

Dr. Ahlad Kumar

Why Proof of Work Works

1. Cost of lying: rewriting history requires redoing all PoW for that block + successors.
2. Security assumption: honest majority of hash power ($\geq 50\%$).

2. Probabilistically: after 6 confirmations, reversal probability is negligible.

Dr. Ahlad Kumar

Summary

1. Byzantine Generals Problem: agreement despite traitors.
2. Bitcoin's solution: Proof of Work makes dishonesty economically infeasible.
3. Honest nodes converge on the longest chain $\rightarrow$ consensus is achieved.

Dr. Ahlad Kumar

Practical Byzantine Fault Tolerance (PBFT)

1. **Introduced by Castro and Liskov (1999)** – a practical solution to the Byzantine Generals Problem.
2. **Permissioned Consensus** – designed for systems with **known validators**.
3. **Fault Tolerance** – tolerates up to f faulty nodes if total $n \geq 3f + 1$.

Dr. Ahlad Kumar

Quantum Computing

Dr. Ahlad Kumar
Blockchain Fundamentals

NFSU Gandhinagar

October 4, 2025

Dr. Ahlad Kumar

What is Quantum Computing?

- A **new model of computation** that uses the laws of quantum mechanics.
- Instead of classical bits (0 or 1), quantum computers use **qubits**.
- Qubits can exist in a state of **0, 1, or both simultaneously** (superposition).
- Quantum computers can process a huge number of possibilities in parallel.
- Goal: Solve problems that are **intractable** for classical computers.
- Examples of applications:
  - Cryptography (breaking RSA/ECDSA).
  - Optimization (logistics, finance).
  - Machine Learning Simulation.

Dr. Ahlad Kumar

Key Quantum Principles

1. **Superposition** – a qubit can be $|0\rangle$, $|1\rangle$, or a combination of both.
2. **Entanglement** – qubits can be correlated; measuring one affects the other, even if far apart.
3. **Interference** – probability amplitudes can add or cancel out, allowing certain outcomes to be amplified.
4. **Quantum Speed-up** – algorithms like:
   - **Shor's Algorithm** – breaks public-key cryptography.
   - **Grover's Algorithm** – accelerates brute-force search.
5. These principles make quantum computers **powerful but also disruptive**.

Dr. Ahlad Kumar

Role of Quantum Computing in Breaking Blockchain Security

Dr. Ahlad Kumar
Blockchain Fundamentals

NFSU Gandhinagar

October 4, 2025

Dr. Ahlad Kumar

Introduction

- Blockchains rely on **classical cryptography** for security.
- Quantum computing introduces a **new class of threats**.
- Key areas affected:
  - Digital Signatures (ECDSA in Bitcoin/Ethereum).
  - Hash Functions (SHA-256, Keccak-256).
  - Consensus fairness and mining power distribution.

Dr. Ahlad Kumar

Cryptographic Threats

1. Public-Key Vulnerability

- Wallets use **Elliptic Curve Digital Signature Algorithm (ECDSA)**.
- **Shor's Algorithm** can efficiently solve discrete logarithms.
- This means private keys can be derived from exposed public keys.
- Impact: **Wallets can be drained** once the public key appears on-chain.

2. Hash Function Threats

- Mining depends on solving hash puzzles (SHA-256 or Keccak-256).
- **Grover's Algorithm** gives quadratic speed-up in brute-force search.
- Result: Reduced mining difficulty, imbalance in competition.

Blockchain Security Impacts

- **Wallet Theft Risk**: Adversaries can compute private keys, stealing funds.
- **Transaction Forgery**: Ability to fake signatures $\rightarrow$ unauthorized transfers.
- **Mining Monopoly**: Quantum miners solve puzzles faster, centralizing power.
- **Consensus Breakdown**: If one party dominates, they can:
  - Perform 51% attacks with fewer resources.
  - Rewrite blockchain history and double spend.

Dr. Ahlad Kumar

Countermeasures (Quantum-Safe Drives)

- **Post-Quantum Cryptography (PQC)**:
  - Lattice-based, hash-based, or multivariate polynomial algorithms.
  - Standardization ongoing by NIST.
- **Quantum-Resistant Signatures**:
  - CRYSTALS-Dilithium, Falcon, SPHINCS+.
- **Hash Agility**: Blockchains must allow future upgrades of hash functions.
- **Hybrid Schemes**: Combine classical + PQC signatures for gradual migration.
- **Quantum Monitoring**: Actively track quantum progress and readiness.

Dr. Ahlad Kumar

Summary

- Quantum computing poses an **existential threat** to blockchain security.
- **Public keys, transactions, and mining processes** are all vulnerable.
- Risk may not be immediate but could escalate quickly.
- **Action is needed now:** adopt quantum-resistant algorithms, prepare migration paths, and design blockchains with crypto-agility.

Dr. Anilad Kumar

What are Memory-Hard Algorithms?

- Algorithms that require a **large amount of memory** to run efficiently.
- Goal: Reduce the advantage of specialized hardware (ASICs/GPUs).
- Key property: Not just CPU-intensive, but also **memory-intensive**.
- Common use-cases:
  - Cryptocurrencies (ASIC-resistance).
  - Password hashing (secure storage).
- Examples: **scrypt, Equihash, Argon2, Ethash**.

Dr. Ahlad Kumar

Example: Buckets of Water Analogy

- Two people race to **fill a swimming pool**.
- **Person A** has a super-fast motor pump (ASIC).
- **Person B** has a normal bucket (CPU).
- If speed is the only factor $\rightarrow$ Person A always wins.

- New Rule: Pool must be filled by carrying **cups of water** from a distant well.
- Pump is useless $\rightarrow$ both must make many trips, needing time and storage.
- This levels the playing field.

Memory-hard algorithms force everyone to use lots of memory, reducing ASIC advantage and promoting fairness.

Dr. Ahlad Kumar

Role of Memory-Hard Algorithms in Blockchain

- Many blockchains use them to make mining **fairer and more decentralized**.
- Prevents mining domination by specialized ASIC hardware.
- Examples:
  - **Bitcoin:** Uses SHA-256 (not memory-hard, ASIC dominated).
  - **Litecoin:** Uses scrypt (memory-hard, initially ASIC-resistant).
  - **Zcash:** Uses Equihash (memory-hard).
  - **Ethereum (pre-PoS):** Used Ethash (partially memory-hard).
- Outside blockchain: used in **password hashing** (e.g., Argon2).
- **Summary:** Memory-hard PoW = slows down ASIC advantage, promotes decentralization.

Dr. Ajay Kumar

Hashcash and its Role in Blockchain

- Proposed by **Adam Back (1997)** to fight email spam and DoS.
- Sender must perform **computational work**:
  - Find a nonce so that hash(message + nonce) has leading zeros.
  - Hard to find, but **easy to verify**.
- **Bitcoin's Proof-of-Work** is based on Hashcash.
  - Miners try different nonces until block hash meets difficulty target.
  - Ensures block creation is **costly**, prevents spam/attacks.
- **Role in Blockchain:**
  - Provides security and decentralization.
  - Prevents spam/DoS on the blockchain.
  - Creates consensus by making block production expensive but verifiable.

Dr. Ahlad Kumar